

Fundamentals of Neural Networks

CH 6

What is a Neural Network?

A neural network is a computational mathematical model inspired by the human brain. Just like our brain has neurons that transmit signals through each other, neural networks have nodes or artificial neurons arranged in layers. Each node is interconnected to other nodes through weighted links.

Data Flow : Data enters the network, gets transformed by mathematical operations, and flows through multiple layers until the output is produced.

Components of Neural Network

1. Neurons

Neurons are the fundamental computational units of neural networks. A neuron receives inputs, multiplies them by weights, adds bias, applies an activation function, and performs a forward pass to share information.

At the end of training, each neuron stores learned parameters like weights and bias — this is its learned knowledge.

2. Role of Layers

i) Input Layer

The main job of this layer is to pass the input values into the network. Input neurons hold and pass the input values but do not participate in computation.

ii) Hidden Layers

These layers perform transformations and calculations on the incoming input values:

- Multiply inputs with corresponding weights
- Add bias
- Apply activation function
- Pass results to next layer

The forward pass produces runtime results, but after training, each neuron has learned weights and biases, which is the ultimate goal.

iii) Output Layer

This layer does the final prediction. The number of neurons depends on the task:

- **Classification** → one neuron per class (with softmax)
 - **Regression** → usually one neuron (continuous value)
-

3. Weights

Weights are numerical values assigned to edges. They are parameters that control the importance or strength of connection between two neurons. Weights are coefficients that multiply input values before summation.

Conceptually: “How much should I care about this input?”

Impact of weight values:

- If w_i is large → that input strongly influences the neuron
- If w_i is near zero → the input is almost ignored
- If w_i is negative → the input has an inhibitory effect

Weights control the slope of the decision boundary.

Weight Expression in a Neuron

For a neuron with inputs $x_1, x_2, \dots, x_n$ and corresponding weights $w_1, w_2, \dots, w_n$:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \sum_{i=1}^n w_ix_i + b$$

Updating Weights During Training

Weights are updated using gradient descent:

$$w_i \leftarrow w_i - \eta \frac{\partial L}{\partial w_i}$$

Where:

- η = learning rate
 - $\frac{\partial L}{\partial w_i}$ = gradient of loss with respect to weight w_i
-

4. Bias [Baseline Offset]

Bias is initialized with small random values. If after calculating weights $\times$ inputs the final value comes to zero, it deactivates our neuron and no learning occurs. That's why we introduce a trainable parameter (bias) that helps the neuron learn even if the weighted sum is zero, providing more flexibility.

Bias controls the position of the decision boundary (while weights control the slope).

Bias lets the neuron represent any line (or hyperplane), not just ones anchored at the origin.

Bias Expression

$$z = \sum_{i=1}^n w_ix_i + b$$

The bias b shifts the activation threshold.

Updating Bias During Training

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Where $\frac{\partial L}{\partial b}$ is the gradient of loss with respect to bias.

5. Activation Function [Bends the Line]

If we stack layers without activation functions, the composition is still linear and represents a straight line. Activation functions introduce non-linear transformations, giving the network the ability to learn complex functions, curves, boundaries, and intricate patterns.

Linear vs Non-Linear Example

- **Linear model (without activation):**

$$y = w_1x_1 + w_2x_2 + b$$

This can only model straight-line relationships, like:
“House price increases by \$100 per square foot”

- **Non-linear model (with activation like ReLU):**

$$h = \max(0, w_1x_1 + w_2x_2 + b)$$

$$y = w_3h + b_2$$

This can model complex patterns, like:

“House price increases slowly at first, then rapidly for larger homes, and plateaus for mansions due to limited buyers.”

Real-world example:

- **Linear:** Salary = \$50,000 + (5,000 × years of experience)
- **Non-linear with activation:** Salary grows slowly initially, then accelerates mid-career, then plateaus at senior levels — a curve that better matches reality.

6. Forward Pass

Forward Pass

The forward pass is the process in a neural network where input data flows layer by layer: each neuron computes a weighted sum of inputs, adds a bias, applies an activation function, and passes the result forward until the output layer produces the final prediction.

Short Example

Suppose we want to classify whether a point (x_1, x_2) belongs to Class A or B:

- Input layer: takes (x_1, x_2)
- Hidden layer neuron:

$$z = w_1x_1 + w_2x_2 + b$$

Apply ReLU:

$$h = \max(0, z)$$

- Output layer neuron:

$$y = \sigma(w_3h + b_2)$$

→ gives probability of Class A vs B

7. Backward Pass

Backward Pass

The backward pass is the process where the network learns: after the forward pass produces an output, the error (loss) is calculated and propagated backward through the layers using the chain rule of calculus. This computes gradients for each weight and bias, which are then updated to reduce future errors.

Short Example

Suppose a simple network predicts $y_{\text{pred}} = 0.8$ for a true label $y_{\text{true}} = 1.0$:

- Forward pass: Input flows through weights, bias, and activation → output = 0.8
- Loss: Error = $(1.0 - 0.8)^2 = 0.04$

- Backward pass: Gradients $\frac{\partial L}{\partial w}$, $\frac{\partial L}{\partial b}$ are computed layer by layer
- Update: Weights and bias are nudged slightly to reduce the error next time

In short: Forward pass = prediction, Backward pass = learning

8. Optimization Function

Optimization in Neural Networks (Definition)

An optimization function (optimizer) is the algorithm that updates the weights and biases of a neural network using the gradients computed during the backward pass, with the goal of minimizing the loss function.

In short: Forward pass → prediction, Backward pass → gradients, Optimization → parameter updates to reduce error

Gradient and Direction

Gradient = partial derivative of the loss with respect to a parameter (weight or bias):

$$g = \frac{\partial L}{\partial \theta}$$

It tells us the slope of the loss surface at the current parameter value.

Direction to move:

- If gradient is positive, increasing the parameter increases loss → move downward (subtract)
- If gradient is negative, decreasing the parameter increases loss → move upward (add)

Update rule (Gradient Descent):

$$\theta \leftarrow \theta - \eta g$$

where η = learning rate (step size)

Always move opposite to the gradient to reduce loss (downhill on the loss surface)

9. Stopping Criteria

Even if optimization keeps reducing loss, we need stopping criteria:

- **Fixed Epochs:** Train for a set number of passes over the dataset (e.g., 50 epochs).
Simple, but may stop too early or waste time
- **Loss Threshold:** Stop when loss is below a chosen value (e.g., $L < 0.01$)
Ensures the model reaches a target accuracy/error
- **Early Stopping (Validation Monitoring)**
Track loss/accuracy on a validation set
Stop when validation loss stops improving or starts increasing while training loss keeps decreasing → sign of overfitting
This is the most practical and widely used method

Example

Suppose we train a classifier:

- Epoch 1: Training loss = 0.50, Validation loss = 0.52
- Epoch 10: Training loss = 0.20, Validation loss = 0.25
- Epoch 20: Training loss = 0.10, Validation loss = 0.35

At Epoch 20, we stop — because validation loss is rising (overfitting), even though training loss keeps decreasing.

10. Loss Function

Loss Function

A loss function is a mathematical measure of how far a neural network's prediction is from the true target. It quantifies the error so the optimizer knows what to minimize during training.

Example: Mean Squared Error (MSE)

Scenario: Predicting house prices (regression task)

- True price: $y_{\text{true}} = \$500,000$

- Predicted price: $y_{\text{pred}} = \$480,000$
- MSE Loss:

$$L = (y_{\text{true}} - y_{\text{pred}})^2 = (500,000 - 480,000)^2 = 400,000,000$$

Or for normalized values:

$$L = (1.0 - 0.96)^2 = 0.0016$$

The optimizer uses this loss value to compute gradients and update weights to make future predictions closer to the true values.

Common Loss Functions

- Regression: Mean Squared Error (MSE), Mean Absolute Error (MAE)
 - Binary Classification: Binary Cross-Entropy
 - Multi-class Classification: Categorical Cross-Entropy
-

Why Shift to Deep Learning from Traditional Machine Learning?

Advantages:

- Works well for massive datasets
- Does not care whether the data is structured or not
- Automatically detects features in the dataset

Limitations:

- Massive computation requirements
- Requires GPU for practical training